Faculty of Engineering and Technology
Electrical and Computer Engineering Department

ARTIFICIAL INTELLIGENCE
ENCS 3340

Course Project No.2:

Text Preprocessing, Feature Engineering, and Machine Learning Model Evaluation for Sentiment Analysis in Arabic Social Media Texts

Prepared by:
Islam Zayed 1230007
Batol Abu Samhadana 1230738

Instructor: Dr. Yazan Abu Farha

Section: 1
Date: January 25, 2026

Table of Contents:

1. Introduction: 4
2. Dataset: 4
    2.1. Data Source: 4
    2.2. Class Distribution: 5
3. Data Preprocessing: 5
    3.1. Text Cleaning: 5
    3.2. Normalization: 5
    3.3. Stopword Elimination: 6
    3.4. Stemming: 6
    3.5. Emoji and Emoticon Replacement: 6
    3.6. Negation Handling: 6
4. Feature Engineering: 7
    4.1. Automated Text Representations: 7
    4.2. Hand-Engineered Features: 7
5. Model Development: 10
    5.1. Data Partitioning: 10
    5.2. Handling Class Imbalance: 10
    5.3. Classifiers: 10
        5.3.0. Baseline Classifier: 11
        5.3.1. Decision Tree Classifier: 11
        5.3.2. Random Forest Classifier: 12
        5.3.3. Naive Bayes Classifier: 13
        5.3.4. MLP Neural Network Classifier: 14
6. Hyperparameter Optimization: 15
    1. Decision Tree: 15
    2. Random Forest: 15
    3. Naive Bayes: 16
    4. MLP Neural Network: 16
7. Model Evaluation: 18
    7.0. Baseline Model: 18
    7.1. Decision Trees: 19
    7.2. Random Forests: 19
    7.3. Naive Bayes: 20
    7.4. Neural Networks: 21
8. Experiment Results and Comparative Analysis: 21
9. Challenges and Mitigation Strategies: 22
    9.1. Class Imbalance: 22

9.2. Arabic Linguistic Complexity: 22
9.3. High Computational Complexity: 23
Conclusion 23
References 24

1. Introduction:

As social media increasingly influences online communication and reflects public opinion, sentiment analysis may be employed to detect various trends, including societal attitudes, brand perceptions and political discourse.

Arabic sentiment analysis of social media data is observed to be particularly challenging due to its:

- Extensive dialectical variation and lack of standard spelling
- Orthographic inconsistency along with noisy, informal text
- Presence of sarcasm, negation and implicit sentiment

This project outlines the development of a comprehensive Arabic Sentiment Analysis system, implemented to classify Arabic tweets into three distinct categories: positive (POS), negative (NEG), and objective/neutral (OBJ). By adopting a hybrid architecture, the framework integrates automated text representations, hand engineered features and word embeddings to train various machine learning classifiers. The performance of the models is rigorously evaluated on a held out test set to ensure accuracy.

2. Dataset:

2.1. Data Source:

The original dataset was provided as a plain text file. It comprises Arabic tweets sourced from various social media platforms. Each entry has been manually annotated with its ground truth sentiment, categorized as one of the following sentiment classes:

- **POS** (Positive): Indicates a favourable sentiment such as expressions of satisfactions
- **NEG** (Negative): Indicates an unfavorable sentiment, including criticism and discontent
- **OBJ** (Objective/Neutral): Indicates factual statements and neutral commentary
- **NEUTRAL**: Merged with OBJ for analytical consistency

We converted the dataset into a structured CSV format by separating the tweet content from its corresponding label and storing them in two columns, namely “text” and “label”.

2.2. Class Distribution:

An initial exploratory data analysis was conducted to understand the structure and distribution of the dataset. The analysis revealed that the corpus consists of 10,006 tweets and that the dataset demonstrates a significant class imbalance, with the distribution as follows:

Table 2.1: *Sentiment Class Distribution of the Dataset Both Pre- and Post- Preprocessing*

Sentiment Label	Count
OBJ	7,523
NEG	1,684
POS	799

The class distribution is visualized in Figure I.1 (Appendix I), highlighting the significant imbalance within the dataset. Accounting for more than half of the samples, the Objective class dominates the dataset. This imbalance may bias machine learning models toward the majority class, potentially leading to poor generalization on minority classes. Therefore, appropriate techniques such as class weighting were implemented in model training.

3. Data Preprocessing:

Arabic social media text requires extensive preprocessing to mitigate noise and enhance performance of the machine learning models. The preprocessing procedure comprises the following stages:

3.1. Text Cleaning:

- Removal of URLs and HTML tags
- Removal of numeric characters
- Removing non-Arabic characters
- Removal of hashtags while preserving their lexical content

3.2. Normalization:

- Normalization of word endings (ه → ة ,ي → ى)
- Unification of Arabic letter variants (ا ← ااإآ)

- Removal of character elongation (ex., "جميل" ← "جمييييل")

3.3. Stopword Elimination:

- Removal of frequent Arabic stopwords downloaded from the NLTK library while retaining negation terms to preserve sentiment polarity. To retain negation terms, a custom exclusion list containing negation terms was applied to the stopwords as follows:

```
# Arabic stopwords from NLTK
nltk.download('stopwords')
arabic_stopwords = set(stopwords.words('arabic'))
# Remove negation words from stopwords
negations = {"غير" ,"بدون" ,"لن" ,"لم" ,"لا" ,"ليس" ,"ما" ,"مو" ,"مش"}
arabic_stopwords = arabic_stopwords - negations
```

3.4. Stemming:

- Application of the ISRI stemming to reduce lexical variants to their canonical roots

3.5. Emoji and Emoticon Replacement:

- Mapping of emojis to sentiment tokens as follows:
  - **EMO_POS**
  - **EMO_NEG**
  - **EMO_NEU**

3.6. Negation Handling:

- Prefixing words immediately following negation terms with NOT_ to capture sentiment inversions

It is evident that the preprocessing stage is fundamental to the system's performance, as it ensures that disparate textual inputs are converted into a standardized format. By mitigating the inherent noise found in Arabic social media text, this process significantly reduces lexical sparsity and minimizes feature dimensionality. Consequently, greater model interpretability is obtained, allowing classifiers to focus on the underlying semantic patterns rather than orthographic variations.

4. Feature Engineering:

4.1. Automated Text Representations:

1. **TF-IDF** (Term Frequency - Inverse Document Frequency): This vectorization method captures the relative importance of terms within the corpus by emphasizing discriminative words while downweighing frequently occurring tokens. To capture both individual word significance and local contextual patterns, a combination of unigrams and bigrams were employed. The vectorizer was trained on the training set only, using:
   - ➔ Unigrams and bigrams: To capture both word-level sentiment and short contextual patterns, improving predictive power.
   - ➔ Maximum of 3,000 features: Reduces dimensionality to speed up training and decrease memory usage. Rare terms don't tend to add noise as only the most informative words are kept, as usually high TF-IDF terms are most sentiment-relevant.
   - ➔ Minimum document frequency of 5: Excluding words that appear in fewer than 5 tweets avoids overfitting as they are likely typos or noise.
   - ➔ Maximum document frequency of 85%: Filters out overly common words while keeping meaningful ones that appear in many tweets but are sentiment-relevant.
2. **Word Embeddings** (FastText): To capture deep semantic relationships, words are projected into a dense vector space using FastText. Unlike traditional embeddings, FastText accounts for subword information making it particularly effective for the Arabic language, given its extensive word-form variation. Tweet-level representations are generated through mean pooling, where the vectors of all constituent tokens are averaged to produce a single, fixed length feature vector for each input, ensuring a consistent representation for the classification models.

4.2. Hand-Engineered Features:

In addition to standard vectorization, the framework utilizes a hybrid feature set to capture linguistic indicators specific to Arabic as summarized Table 2 below.

Table 4.1: *Hand-Engineered Linguistic and Structural Features*

Feature	Description
tweet_length	Number of tokens per tweet. Captures verbosity and emphasis. Extremely short tweets often express strong opinions, while longer ones may contain neutral information.
punctuation_count	Total punctuation marks. Punctuation marks (especially "!" and "?) are frequently used to convey emphasis or emotional intensity, correlating with subjective sentiment in informal text.
char_elongation_count	Repeated characters per tweet. Character elongation is common in Arabic social media to emphasize emotion, often indicating strong positive or negative sentiment.
word_repetition_count	Sequentially repeated words. Repeated words reflect emphasis or emotional reinforcement which is typically associated with strong sentiment polarity.
hashtag_count	Number of hashtags. Hashtags often summarize the topic or sentiment of a tweet and may encode stance or emotion explicitly, contributing contextual sentiment information.
emoji_count	Number of emojis. Emojis serve as direct indicators of affective state and can disambiguate sentiment when textual cues are weak or ambiguous.
emoticon_count	Repeated emoticons. Similar to emojis, emoticons are explicit sentiment markers.
neg_count	Count of negation words. Negation words significantly affect sentiment polarity by reversing or weakening emotional meaning,

	making negation frequency critical for correct sentiment interpretation.
dialect	Presence of dialectal terms. Dialectal Arabic terms capture regional and informal usage that is often sentiment rich and poorly represented in Modern Standard Arabic resources.
has_positive_emoji	Binary indicator for positive emojis. Explicitly captures the presence of positive affect, providing a strong, low-noise sentiment signal.
has_negetive_emoji	Binary indicator for negative emojis. Similar to positive emoji detection, this feature captures explicit negative affect, improving class separability.
repeated_punt_count	Repeated punctuation sequences. Signals heightened emotional intensity and urgency, correlating with strong sentiment expression.

5. Model Development:

5.1. Data Partitioning:

The corpus was partitioned into training, validation, and test sets using a stratified sampling technique. This approach was critical to ensure that the inherent class imbalance present in the original data was preserved proportionally across all subsets. By maintaining these ratios, the framework facilitates robust model convergence during training, and guarantees an unbiased, statistically significant evaluation of the system's performance on unseen data. The size of the sets required for the machine learning models were proportioned as follows:

- 60 % training
- 20 % validation
- 20 % testing

5.2. Handling Class Imbalance:

To counter the skew toward the majority class, a balanced weighting schema was adopted. This ensured that the minority sentiment classes exerted a proportional influence on model optimization, specifically through class-weight adjustment in tree-based algorithms and weighted loss functions with the MLP. The implementation using the Scikit-Learn utility is detailed below:

```
from sklearn.utils.class_weight import compute_class_weight
import numpy as np
classes = np.unique(labels_train)
class_weights                                                                    =
compute_class_weight(class_weight="balanced",classes=classes,y=labels_train)
```

5.3. Classifiers:

In this stage, we trained and validated multiple classical machine learning models and a shallow neural network for Arabic sentiment classification. The goal was to compare different learning paradigms under the same preprocessing, feature extraction, and data splitting strategy, while addressing key challenges such as class imbalance.

All models were trained on the training set (60%). We **tuned and evaluated them on the validation set (20%),** and later assessed them on the held-out test set (20%). We selected the Macro F1-score as the primary optimization and comparison metric, as it treats all sentiment classes equally and is more appropriate for imbalanced datasets than accuracy alone.

5.3.0. Baseline Classifier:

We started with a **Dummy Classifier** that always predicts the most frequent class (Objective/Neutral). This served as a **baseline** to highlight the impact of class imbalance.

```
--- Baseline (Majority Class) (Validation) ---
Accuracy: 0.7521239380309845
              precision    recall  f1-score   support

         NEG       0.00      0.00      0.00       337
         OBJ       0.75      1.00      0.86      1505
         POS       0.00      0.00      0.00       159

    accuracy                           0.75      2001
   macro avg       0.25      0.33      0.29      2001
weighted avg       0.57      0.75      0.65      2001
```

Figure 5.1: *Baseline Evaluation on the Validation set*

Although the baseline achieved a relatively high accuracy as we can see from the figure due to the dominance of the Objective class, it completely failed to identify Positive and Negative sentiments, resulting in a **very low Macro F1-score**. This confirmed that **accuracy alone is misleading**, reinforcing the need for more sophisticated models.

5.3.1. Decision Tree Classifier:

We trained a **Decision Tree** to capture non-linear relationships between features. Decision trees naturally handle feature interactions and don't assume linear separability.

Hyperparameter Tuning Approach

At first, we **picked hyperparameters manually**, adjusting them while keeping an eye on the validation set. This gave us good initial performance without relying on expensive computation. Later, we used **GridSearchCV** around the manual values to **fine-tune** the model and find the best combination.

The parameters we considered included:

- Maximum depth ∈ {10, 15, 20}
  Minimum samples per split ∈ {2, 5, 10}
- Minimum samples per leaf ∈ {1, 2, 5}

We also applied **class weights** to reduce bias toward the majority (Objective) class.

Observations:

- The final tree with **max depth = 10** gave the best balance between complexity and validation performance.
- Grid search helped confirm our manual choices but didn't drastically change performance.

5.3.2. Random Forest Classifier:

We followed a similar approach as the Decision Tree in the Random Forest:

1. **Manual tuning first:** We adjusted parameters based on the validation set performance.
2. **GridSearchCV:** We searched around the manually chosen values for further fine-tuning.

The parameters included:

- Number of trees: {100, 200}
- Maximum depth: {10, 15, 20}
- Minimum samples per split: {2, 5}
- Minimum samples per leaf: {1, 2}

```
--- Random Forest (Validation) ---
Accuracy: 0.6101949025487257
              precision    recall  f1-score   support

         NEG       0.31      0.48      0.37       337
         OBJ       0.81      0.67      0.73      1505
         POS       0.23      0.32      0.27       159

    accuracy                           0.61      2001
   macro avg       0.45      0.49      0.46      2001
weighted avg       0.68      0.61      0.63      2001
```

Figure 5.2: *Random Forest Parameter Tuning Evaluation on the Validation set*

Observations:

- The grid search confirmed our manual choices and slightly improved some metrics, but the overall gains were limited.

5.3.3. Naive Bayes Classifier:

We used **MultinomialNB** because it works well with TF-IDF features.

- Initial manual tuning gave reasonable performance.
- GridSearchCV helped us select the best **alpha** and **fit_prior** values.

```
--- Naïve Bayes (Validation) ---
Accuracy: 0.7401299350324838
              precision    recall  f1-score   support

         NEG       0.41      0.22      0.29       337
         OBJ       0.78      0.92      0.85      1505
         POS       0.40      0.13      0.19       159

    accuracy                           0.74      2001
   macro avg       0.53      0.42      0.44      2001
weighted avg       0.69      0.74      0.70      2001
```

Figure 5.3: *Naive Bayes Parameter Tuning Evaluation on the Validation set*

Observations:

- ➢ Naïve Bayes performed well in the Objective class but struggled with Positive and Negative classes.

  The Macro F1 improved with grid search, confirming the value of systematic tuning.

5.3.4. MLP Neural Network Classifier:

We trained a **shallow feed-forward neural network** with two hidden layers (128, 64) using ReLU activations and Adam optimizer.

Hyperparameter Tuning Approach

Neural networks have **high computational complexity**, with a single model taking several minutes to train. Running a full GridSearchCV over multiple networks would take **hours**, which was impractical.

- ➢ We **manually selected hyperparameters** based on validation set performance, observing changes and adjusting iteratively.
- ➢ We prioritized practical performance without excessive computation.

```
--- MLP Neural Network (Validation) ---
Accuracy: 0.7041479260369815
              precision    recall  f1-score   support

         NEG       0.35      0.28      0.31       337
         OBJ       0.79      0.86      0.82      1505
         POS       0.28      0.14      0.19       159

    accuracy                           0.70      2001
   macro avg       0.47      0.43      0.44      2001
weighted avg       0.67      0.70      0.68      2001
```

Figure 5.4: *Neural Network Parameter Tuning Evaluation on the Validation set*

Observations:

- ➢ Manual tuning gave good Macro F1 while keeping training time manageable.
- ➢ Validation set monitoring allowed us to balance complexity and accuracy effectively.

6. Hyperparameter Optimization:

Hyperparameter optimization was conducted using a guided trail-and-error approach, where parameter ranges were selected based on standard defaults and practical considerations. [1] The optimal values were chosen according to validation macro-F1 performance as follows:

1. **Decision Tree:**
   a. Tuned maximum depth (None, 10, 20, 30) and minimum samples per split (2, 5, 10). In maximum tree depth, None allows the tree to grow until all leaves are pure, providing a baseline. 10, 20, and 30 are typical depths for text classification, as shallow trees prevent overfitting, though deeper trees capture more complexity. In regards to minimum sample splits, smaller numbers allow for more splits (therefore more complexity), whereas larger numbers help reduce overfitting.
   b. For each combination, the model was trained on the training set and evaluated on the validation set using macro F1-score
   c. The combination achieving the highest F1-score was selected as the final model's hyperparameters

```
=== Decision Tree Best Params (Validation) ===
max_depth=None, min_samples_split=5
```

2. **Random Forest:**
   a. Tuned number of trees (100, 200):
      - More trees usually tend to improve stability, but at the expense of increased computation. 100-200 provides enough trees to stabilize performance without excessive runtime.
   b. Maximum depth (None, 10, 20):

        - Deeper trees may capture complex patterns but at risk of overfitting
    c. Minimum samples per split (2,5):
        - Balances overfitting vs. underfitting, as simple ranges avoids overly fined tuned nodes while keeping flexibility
    d. Each configuration was trained on the training set and validated on the validation set, selecting the configuration with the best macro F1-score.

```
=== Random Forest Best Params (Validation) ===
n_estimators=200, max_depth=20, min_samples_split=5
```

3. **Naive Bayes:**
    a. Tuned smoothing parameter α (0.1, 0.3, 0.5, 1.0) using validation F1-score
    b. The α value giving the best performance was used for the final model

```
=== Naïve Bayes Best Alpha (Validation) ===
alpha=1.0
```

4. **MLP Neural Network:**
    a. Tuned hidden layer sizes (128, (128,64)):
        - Controls model capacity
        - Single layer vs. two layers allows testing shallow vs. moderately deep networks
    b. Learning rates (0.001, 0.0005) while handling class imbalance through sample weights.
        - The step sizes for weight updates affects convergence speed and stability
        - 0.001 is a standard default for Adam optimizer, as 0.0005 is slightly smaller to test more stable convergence.
    c. Each configuration was trained on the training set and evaluated on the validation set. The best F1-score determined the final architecture and learning rates

```
=== MLP Best Params (Validation) ===
hidden_layers=(128, 64), learning_rate=0.001
```

At the start, we experimented with **hyper parameters manually,** picking values that we thought would work well. We adjusted them gradually while **monitoring the validation set performance** to see whether the changes improved or worsened the results. This gave us a good initial sense of what settings were effective.

In the very beginning, **we did not use GridSearchCV.** Even with this manual approach, we were able to get okay results on the validation set, which helped us establish a baseline for fine-tuning.

Afterward, we decided to use **GridSearchCV** models to systematically explore hyperparameter combinations around the values we had already found manually. This allowed us to **fine-tune the models** and push their performance as high as possible.

However, some models, especially **neural networks,** have very high computational complexity. Training a single network can take several minutes, so running GridSearchCV over 20+ configurations **would take hours.** To balance performance and efficiency, we kept the hyperparameters we selected manually for these models, and explored options close to them, using the validation set results to **guide our choices.** This approach lets us achieve strong performance **without excessive computation time.**

7. Model Evaluation:

After training and fine-tuning the models, we evaluated them based on the **hold-out test set,** to evaluate their performances. We focused primarily on the **Macro F1-Score,** because it accounts for all classes equally, which is crucial given the **class imbalance** in our dataset. We also reported **accuracy, precision** and **recall** for completeness.

7.0. Baseline Model:

```
============================================================
Baseline Model — Test Set Evaluation
============================================================
Accuracy        : 0.7517
F1 Macro (main) : 0.2861
F1 Weighted     : 0.6452

Classification Report:
              precision    recall  f1-score   support

         NEG     0.0000    0.0000    0.0000       337
         OBJ     0.7517    1.0000    0.8583      1505
         POS     0.0000    0.0000    0.0000       160

    accuracy                         0.7517      2002
   macro avg     0.2506    0.3333    0.2861      2002
weighted avg     0.5651    0.7517    0.6452      2002

Confusion Matrix:
[[   0  337    0]
 [   0 1505    0]
 [   0  160    0]]
```

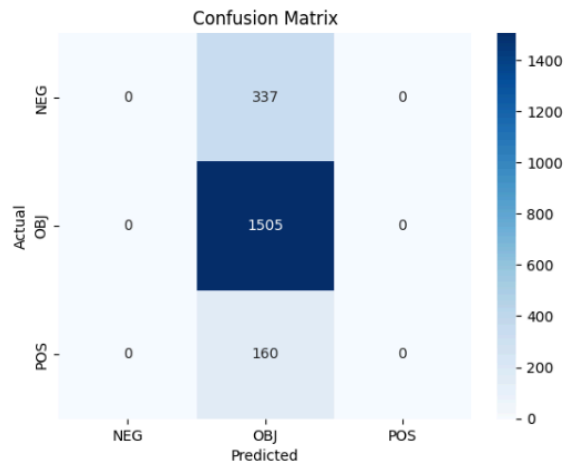

Figure 7.1: *Baseline Confusion Matrix and Evaluation Results on the Test Dataset*

- **Accuracy:** High (Due to the dominance of the objective class)
- **F1 Score:** Very low especially for the POS and NEG classes

This reinforced that accuracy alone can be misleading. Because although the model had the highest accuracy, it completely failed to identify minor classes.

7.1. Decision Trees:

```
============================================================
Decision Tree — Test Set Evaluation
============================================================
Accuracy        : 0.7088
F1 Macro (main) : 0.4305
F1 Weighted     : 0.6784

Classification Report:
              precision    recall  f1-score   support

         NEG     0.3694    0.1721    0.2348       337
         OBJ     0.7756    0.8817    0.8252      1505
         POS     0.2537    0.2125    0.2313       160

    accuracy                         0.7088      2002
   macro avg     0.4662    0.4221    0.4305      2002
weighted avg     0.6655    0.7088    0.6784      2002

Confusion Matrix:
[[  58  266   13]
 [  91 1327   87]
 [   8  118   34]]
```

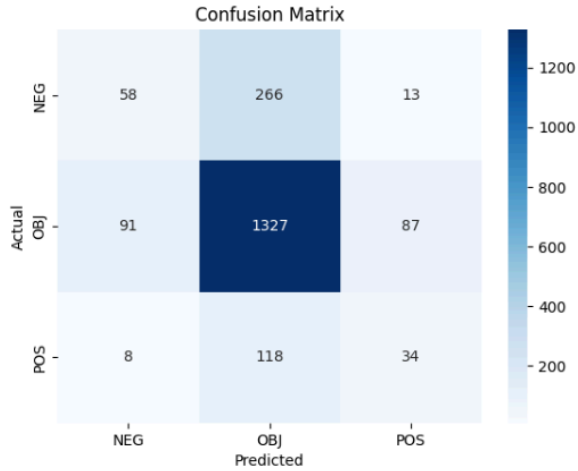

Figure 7.2: *Decision Tree Confusion Matrix and Evaluation Results on the Test Dataset*

- **Performance:** Moderate to strong Macro F1-score
- **Observations:**
  - The tree captured the non-linear relationships between features.
  - Class weighing helped reduce bias towards the OBJ class.
  - GridSearch slightly improved the evaluation metrics.

7.2. Random Forests:

```
============================================================
Random Forest — Test Set Evaluation
============================================================
Accuracy        : 0.6174
F1 Macro (main) : 0.4741
F1 Weighted     : 0.6433

Classification Report:
              precision    recall  f1-score   support

         NEG     0.3277    0.5163    0.4009       337
         OBJ     0.8196    0.6671    0.7355      1505
         POS     0.2358    0.3625    0.2857       160

    accuracy                         0.6174      2002
   macro avg     0.4610    0.5153    0.4741      2002
weighted avg     0.6901    0.6174    0.6433      2002

Confusion Matrix:
[[ 174  139   24]
 [ 337 1004  164]
 [  20   82   58]]
```

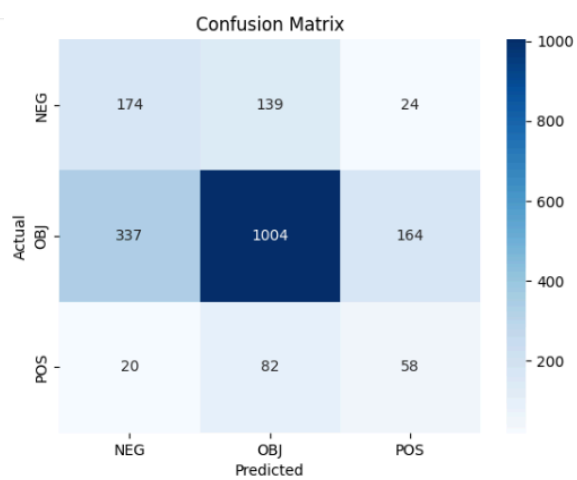

Figure 7.3: *Random Forest Confusion Matrix and Evaluation Results on the Test Dataset*

- **Performance:** Stronger than the single Decision Tree.
- **Observations:**
  - Ensembling multiple trees improved generalization and reduced overfitting.
  - The F1-score improved especially in POS and NEG classes.
  - GridSearch around manually chosen hyper parameters gave minor gains.

7.3. Naive Bayes:

```
============================================================
Naïve Bayes — Test Set Evaluation
============================================================
Accuracy        : 0.7363
F1 Macro (main) : 0.4214
F1 Weighted     : 0.6904

Classification Report:
              precision    recall  f1-score   support

         NEG     0.3987    0.1869    0.2545       337
         OBJ     0.7749    0.9262    0.8438      1505
         POS     0.3778    0.1062    0.1659       160

    accuracy                         0.7363      2002
   macro avg     0.5171    0.4065    0.4214      2002
weighted avg     0.6798    0.7363    0.6904      2002

Confusion Matrix:
[[  63  270    4]
 [  87 1394   24]
 [   8  135   17]]
```

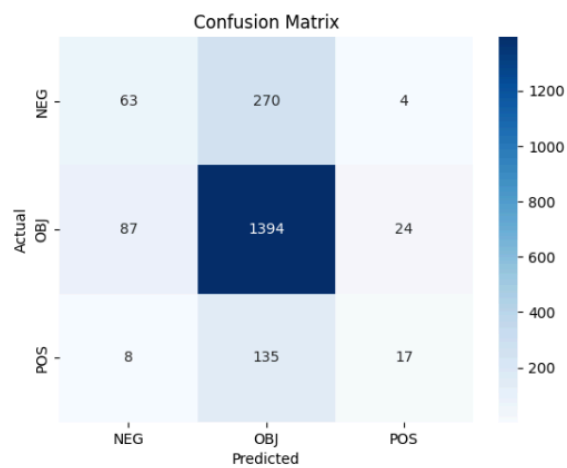

Figure 7.4: *Naive Bayes Confusion Matrix and Evaluation Results on the Test Dataset*

- **Performance:** Relatively weak in minor classes.
- **Observations:**
  - It served as a fast and easy method amongst other models.
  - It worked with sparse TF-IDF Features.
  - GridSearch helped with smoothing alpha.

7.4. Neural Networks:

```
============================================================
MLP Neural Network — Test Set Evaluation
============================================================
Accuracy        : 0.7053
F1 Macro (main) : 0.4532
F1 Weighted     : 0.6876

Classification Report:
              precision    recall  f1-score   support

         NEG     0.3658    0.2789    0.3165       337
         OBJ     0.7873    0.8558    0.8201      1505
         POS     0.2752    0.1875    0.2230       160

    accuracy                         0.7053      2002
   macro avg     0.4761    0.4407    0.4532      2002
weighted avg     0.6754    0.7053    0.6876      2002

Confusion Matrix:
[[  94  229   14]
 [ 152 1288   65]
 [  11  119   30]]
```

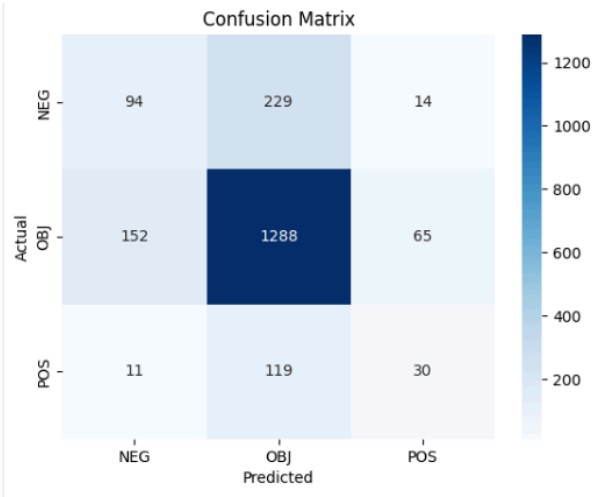

Figure 7.5: *Neural Network Confusion Matrix and Evaluation Results on the Test Dataset*

- **Performance:** Comparable Macro F1 to Random Forest, and slightly better on minority.
- **Observations:**
  - Manual hyper parameter tuning was sufficient to achieve a very good performance.
  - Neural networks capture the complex relations between features, improving prediction for POS and NEG sentiments.

8. Experiment Results and Comparative Analysis:

	Macro F1			Accuracy		
Stage	Initial	Test	Tuned	Initial	Test	Tuned
Model						
Baseline	0.286	0.286	0.286	0.752	0.752	0.752
Decision Tree	0.442	0.430	0.420	0.697	0.709	0.708
MLP Neural Network	0.444	0.453	0.444	0.709	0.705	0.709
Naïve Bayes	0.350	0.421	0.418	0.760	0.736	0.741
Random Forest	0.483	0.474	0.477	0.659	0.617	0.633

Figure 8.1: *Experimental results and Comparative Analysis between the classifiers in various stages*

** Initial: tuned with handpicked hyperparameters based on the val set.
Tuned: tuned with gridsearch, tested on val set.
Test: tuned with gridsearch, tested on test set.

- MLP Neural Network was observed to perform the best in overall accuracy and maintain balanced precision and recall across sentiment classes. Its use of combined feature sets and sample weighting proved to improve detection of minority classes.
- Random Forest outperforms Decision Tree and Naive Bayes, showing that its ensemble method better captures complex patterns in data.
- Naive Bayes achieves a good recall but lower accuracy, often predicting the majority class and misclassifying minority sentiments.
- Across all the models, OBJ tweets dominate misclassifications, possibly due to their high prevalence and linguistic diversity.
- Hand-Engineered features and word embeddings significantly improve performance, especially for the minority classes, by capturing stylistic and semantic nuances in Arabic social media text.
- While the test set was not used in the tuning or training process, all the models still showed great results on it.

9. Challenges and Mitigation Strategies:

9.1. Class Imbalance:

- Applied class weights and sample weighting
- Disabled prior probabilities in Naive Bayes

9.2. Arabic Linguistic Complexity:

- Extensive normalization and stemming
- Dialect-aware features

9.3. High Computational Complexity:

- Neural Networks specifically had very high computation time even with limiting the number of iterations. Because of that we decided to only do manual hyper parameter tuning for it instead of trying GridSearchCV.

Conclusion

After completion of the project, a machine learning based sentiment analysis for Arabic social media text through a comprehensive pipeline consisting of text preprocessing, feature engineering, and model evaluation is presented. Several classifiers, namely Decision Tree, Random Forest, Naive Bayes and a Multi-Layer Perceptron, were trained and evaluated on an imbalanced, multi-class Arabic dataset. Validation-based hyperparameter tuning was employed to select optimal model configurations while avoiding test data leakage. Experimental results demonstrated that ensemble and neural models, particularly the Random Forest and MLP, consistently outperformed simpler probabilistic and tree-based baselines in terms of macro-averaged F1 score, highlighting their ability to better capture complex patterns in high-dimensional sparse text representations. The results confirm that careful preprocessing, appropriate feature selection, and systematic hyperparameter optimization play a critical role in enhancing Arabic sentiment classification performance, especially under class imbalance and dialectal variation.

References

[1] D. Suleiman, A. Odeh, and R. Al-Sayyed, "Arabic sentiment analysis using Naïve Bayes and CNN-LSTM," *Informatica*, vol. 46, no. 6, pp. 79–86, 2022, doi: 10.31449/inf.v46i6.4199.

Appendices

Appendix I:

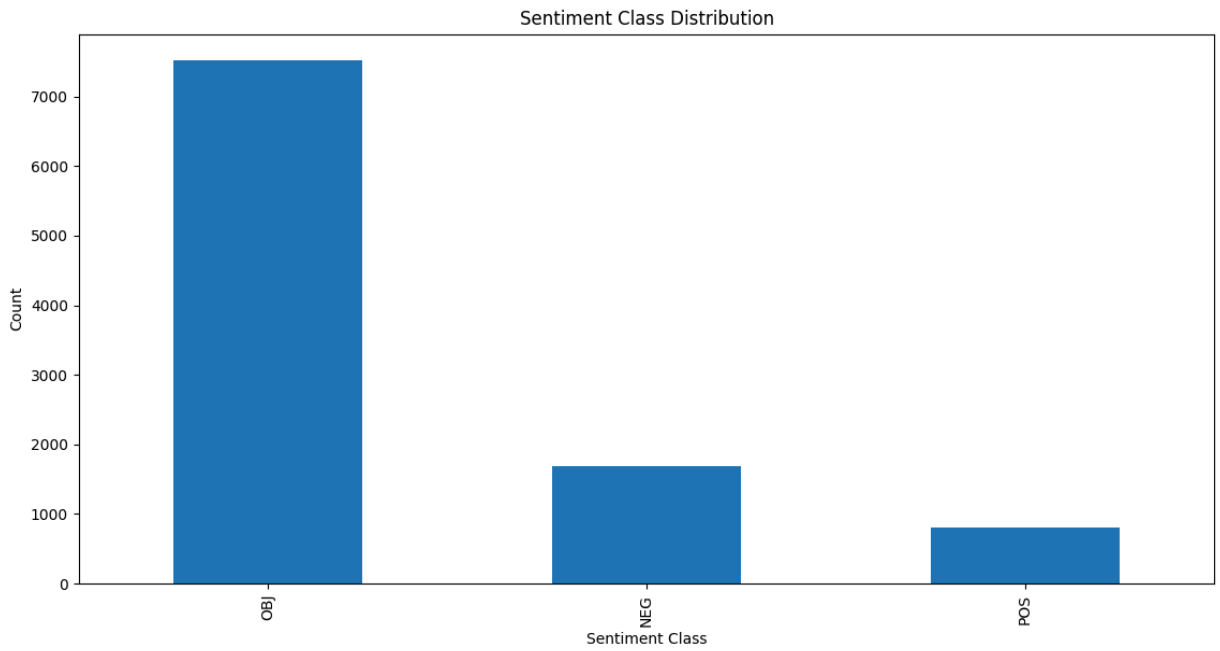

Figure I.1: *Sentiment Class Distribution of the Dataset (Same in Both Pre- and Post- Preprocessing)*

```
C:\Users\PC\PycharmProjects\PythonProject\.venv\Scripts\python.exe
C:\Users\PC\PycharmProjects\PythonProject\Sentiment.py
=== BEFORE CLEANING ===

text label
0                                                                    بعد استقالة رئيس
#المحكمة_الدستورية ننتظر استقالة #رئيس_القضاء #السودان    OBJ
1                                                                    أهنئ الدكتور أحمد جمال الدين،
القيادي بحزب مصر، بمناسبة صدور أولى روايته    POS
2                                                                    البرادعي يستقوى بامريكا
مرةاخرى و يرسل عصام العريان الي واشنطن شئ مقرف    NEG
3                                                                    الحرية_والعدالة / شاهد الآن: #ليلة_الاتحادية أول فيلم استقصائي يتناول أسرار و#
كواليس تعرض لأول مرة حول حقيقة    OBJ
4  الوالدة لو اقولها بخاطري حشيشة تضحك بس من اقولها ملل الله وكيلك تعطيني محاضرة عن الفسق والفجور بجنوب
الشيشان #ليه كذا يانبع الحنان    OBJ
(10006, 2)
label
OBJ    7523
NEG    1684
POS     799
```

```
Name: count, dtype: int64

=== AFTER CLEANING ===
                                                                    cleaned_text
label
0                                                قال رئس حكم دستور تظر قال رئس قضء سود    OBJ
1                                          هنئ دكتور حمد جمل دين قيدى حزب مصر نسب صدر ولى ريت    POS
2                                          رادعى تقى بمر مرةاخرى رسل عصم عري الى شنط شئ قرف    NEG
3  حرة عدل شهد الن ليل تحد اول يلم استقصائى نول سرر كواليس عرض لول مرة حول حقق    OBJ
4   ولد اقل خاطرى حشش تضح اقل مل الة وكل عطينى حضر فسق فجر جنب شيش لية نبع حنن    OBJ
(10006, 3)
label
OBJ    7523
NEG    1684
POS     799
Name: count, dtype: int64

Preprocessed data saved to 'Arabic.txt_preprocessed.csv'
Class weights: {'NEG': np.float64(1.9811881188118812), 'OBJ':
np.float64(0.4433857744294261), 'POS': np.float64(4.16875)}
Training features shape: (6003, 3012)
Validation features shape: (2001, 3012)
Test features shape: (2002, 3012)
=== Decision Tree Best Params (Validation) ===
max_depth=None, min_samples_split=5

=== Random Forest Best Params (Validation) ===
n_estimators=200, max_depth=20, min_samples_split=5

=== Naïve Bayes Best Alpha (Validation) ===
alpha=1.0

=== MLP Best Params (Validation) ===
hidden_layers=(128,), learning_rate=0.001

=== Decision Tree Evaluation on Test Set ===
Accuracy: 0.6244
Precision: 0.4175
Recall: 0.4322
F1-score: 0.4227
Confusion Matrix:
 [[  36   27   97]
 [  28  115  194]
 [ 126  280 1099]]

=== Random Forest Evaluation on Test Set ===
Accuracy: 0.6633
```

```
Precision: 0.4854
Recall: 0.5150
F1-score: 0.4934
Confusion Matrix:
 [[  44   25   91]
 [  14  181  142]
 [ 104  298 1103]]

=== Naïve Bayes Evaluation on Test Set ===
Accuracy: 0.6134
Precision: 0.4557
Recall: 0.5121
F1-score: 0.4672
Confusion Matrix:
 [[ 49  21  90]
 [ 22 194 121]
 [200 320 985]]

=== MLP Neural Network Evaluation on Test Set ===
Accuracy: 0.6833
Precision: 0.4794
Recall: 0.4867
F1-score: 0.4828
Confusion Matrix:
 [[  43   17  100]
 [  19  135  183]
 [ 105  210 1190]]

Process finished with exit code 0
```